What Carries Architectural Signal? A Code-to-Architecture Mapping Study

Rakhshanda Jabeen*†, Morgan Ericsson†, Yuyao Duan†, Jonas Nordqvist†, and Anna Wingkvist†
*Electrolux Professional AB, Ljungby, Sweden
Email: rakhshanda.jabeen@electroluxprofessional.com
†Linnaeus University, Växjö, Sweden
Email: {morgan.ericsson | jonas.nordqvist | anna.wingkvist}@lnu.se

***Abstract*—As software evolves, implementation can gradually diverge from the intended architecture, weakening its modular structure and making the system harder to maintain. Recovering this architecture and mapping implemented files to it are two sides of the same problem: both depend on which architectural signals a file's representation actually carries. We study this through code-to-architecture mapping, formulated as semi-supervised node classification over a heterogeneous graph of folders, files, and members connected by structural and dependency relations. A small set of known file-to-module assignments provides supervision, letting us isolate how each signal contributes to assigning the rest. Existing approaches typically fix how lexical, dependency, and organizational signals are represented, without isolating their individual contributions. Using a Heterogeneous Graph Transformer (HGT), we vary these representations and the graph structure independently, comparing a structural embedding from node2vec, a lexical embedding from Word2Vec, and a contextual embedding from a dependency-guided fine-tuned UniXcoder. Across 100 runs on ten open-source Java, C, and C++ systems, we evaluate each representation on its own and as an HGT initialization. On their own, node2vec and fine-tuned UniXcoder consistently outperform Word2Vec, showing dependency information is the strongest signal when folder organization is absent. Message passing narrows this gap, but Word2Vec still trails the other two, so the ranking holds. Adding folder organization removes most remaining differences and yields near-ceiling performance, though this partly reflects how closely folder layout already tracks the reference decomposition rather than an independently strong signal. Dependency structure remains the more informative signal on its own, bounded by system connectivity and architectural cohesion rather than by proximity to ground truth.**

***Index Terms*—Software Architecture Recovery, Code-to-Architecture Mapping, Heterogeneous Graph Neural Networks, Graph Transformers, Representation Learning**

I. Introduction

Software architecture describes a system's high-level structure in terms of its principal elements, their responsibilities, and the relations among them, and provides the basis for reasoning about maintainability and other quality attributes [1]. This structure is typically reflected in the implementation as a modular decomposition, a partition of files into modules that group related functionality [2]. As a system evolves, developers introduce dependencies that were not anticipated in the original design and add functionality that fits no existing module, while documentation of the intended architecture is rarely kept in step with these changes. The resulting gap between design and implementation is described as architectural drift, where the implementation grows away from the intended design without directly violating it, or architectural erosion, where it does [3].

Recovering or approximating this correspondence depends on what evidence about the architectural role a source code file exposes. A source file carries several distinct kinds of evidence about its architectural role. Static dependencies position it within a structural neighborhood shaped by the system's modular decomposition [4, 5]. Its location in the folder hierarchy reflects how developers organize related functionality and may preserve architectural boundaries even when no formal architecture is documented [6]. Its identifiers and implementation content encode domain concepts and behavioral information that dependencies and folder placement do not directly expose [7].

Prior work has combined such sources manually [7, 8] and through representation learning approaches, but has generally fixed the representation selected for each source rather than comparing alternatives. GAER used a graph autoencoder with Word2Vec identifier embeddings for unsupervised recovery [9], while prior semi-supervised mapping work used lexical and folder-path features with graph convolutional networks [10]. NEGAR used node2vec over a file-level dependency graph to learn structural embeddings for recovery [4]. Later work compared structural representations more directly, evaluating graph attention with diffusion-based rewiring against node2vec and studying the contribution of project organization through alternative graph representations [11]. However, these studies did not compare the available signals under matched conditions or examine how their relative usefulness depends on the graph representation. It therefore remains unclear which signal provides the strongest architectural evidence and how it should be represented.

We study this question through the task of assigning source files to the modules of a reference architecture, which we refer to as code-to-architecture (C2A) mapping[1]. This task provides an evaluation with ground truth and is a prerequisite for reflexion modeling, where implementation-level and architecture-level dependencies are compared to detect drift [12]. We work in the semi-supervised setting, where a small set of files is already mapped, and the task is to infer the remaining

[1]Code and data are available at https://github.com/sse-lnu/hetmap.

assignments. This reflects a setting in which an existing mapping must be extended or updated as the implementation evolves [13]. This setting is well suited to studying input signals, because with few labels available, the choice of how files are represented and what the model is asked to propagate over largely determines what can be learned.

The present paper studies these signals directly. We represent each software system as a heterogeneous graph with three node types: folders, files, and members. Each file is associated with a single member node that aggregates the methods, fields, and other program elements contained within it, rather than representing each program element as a separate node. Structural relations connect folders to their contents and files to their member nodes, while typed dependency relations connect files and members that interact in the code. We learn from this graph using a Heterogeneous Graph Transformer (HGT) [14], which conditions attention and message passing on source node type, target node type, and relation type, letting us vary node features and graph structure independently while holding the propagation mechanism fixed. We compare three node feature types: structural embeddings from node2vec, trained on the dependency graph alone [4, 11]; lexical embeddings from Word2Vec over code identifiers [9]; and contextual code embeddings from UniXcoder [15], adapted through dependency-guided contrastive fine-tuning using positive pairs drawn from the dependency graph. We compare these feature types in isolation and, once they initialize the heterogeneous graph model, vary the available graph structure to separate the contribution of structure from that of the node features themselves.

This paper is structured to address the following research questions:

1) **RQ1.** How does a heterogeneous graph transformer perform on code-to-architecture mapping relative to existing GNN-based approaches?
2) **RQ2.** How much architectural information do structural, lexical, and contextual training signals carry when used as file representations?
3) **RQ3.** Does the relative effectiveness of structural, lexical, and contextual node features persist when they initialize a heterogeneous graph model, and how much does graph structure add beyond each feature type alone?

II. Related Work

Architecture recovery and code-to-architecture mapping both infer a system's modular structure from implementation artifacts, relying on evidence that includes static dependencies, code semantics, and directory organization, among other sources. Recovery produces a decomposition without a reference architecture, while mapping assigns entities to the modules of a given one. This paper studies which of these signals carries architectural information rather than a single task; we organize prior work by information source across both settings.

Architecture recovery techniques have historically been built around one dominant signal. Dependency-based methods treat the system as a graph and partition it to maximize modularization quality, using search-based optimization [16, 17, 18, 19, 20] or clustering over dependency-derived similarity [21, 22, 23, 24, 25, 26]. These perform well when dependencies align with architectural boundaries, but recovery quality is sensitive to how the graph is constructed and how relations are weighted, since shared utilities and layered interactions introduce cross-module edges. Semantic methods instead cluster artifacts by identifiers and comments in a latent space learned from source code [27, 28, 29] or from lexical cues [30], and directory-based methods use file names and paths as an architectural signal [31]. Non-structural signals can be noisy in isolation because reused terminology and utility directories mix entities from many modules. Hybrid methods therefore combine sources through weighted similarity fusion and many-objective remodularization [32, 8, 33, 34], consensus clustering [35], semantic-guided propagation [36], or weighted graphs with community detection [37, 7], with the signals to combine and their relative weight fixed in advance.

Mapping shows the same pattern from a different starting point. Early work framed the assignment of unmapped entities as orphan adoption [38], and HuGMe introduced attraction functions to recommend mappings between entities and modules [39, 40]. Subsequent methods varied the signal and the learner: information-retrieval attraction based on textual similarity [41], supervised classifiers for orphan adoption [42], and a semi-supervised Naive Bayes approach combining lexical and dependency-derived information through Concrete Dependency Abstraction [43, 44]. Interactive and text-based approaches followed, comparing text classifiers for module assignment [45], recommending mappings from natural-language architecture descriptions [46], and integrating package- and class-level granularity [47]. More recent work frames method-level mapping as text classification over TF-IDF, GloVe, and LLM representations [48], examines how textual data properties condition embedding-based classification performance [49], and extracts architectural concepts from documentation and code [50]; LLM-based chain-of-thought prompting has also been used to map implementation units to a reference architecture [51], though such prompting-based approaches face scalability challenges on large codebases given context-size limits and the cost of repeatedly processing implementation artifacts.

Representation learning has also been applied to this problem, but each study fixes the representation rather than comparing alternatives. NEGAR clusters node2vec embeddings from the dependency graph alone, with no node semantics [4]. Jabeen et al. [11] build on this by adding folder organization and comparing node2vec, meta-path walks, and graph attention with and without diffusion rewiring, without lexical features. GAER and Jabeen et al. [9, 52] instead fix folder and identifier tokens into one encoding, Word2Vec-based in the former and contrastively trained with edge attributes in the latter, without comparing it to alternatives. For mapping, prior work evaluated graph neural networks on homogeneous and dependency-type-aware graphs [10], without folder hierarchies, containment, member-level artifacts, or separating graph contribution from

feature contribution. No study compares structural, lexical, and contextual representations under matched conditions, on the same systems and model, isolated from the graph structure they initialize. This is the gap this paper addresses.

III. Background

Software artifacts are naturally relational and multi-typed. A system contains folders, source files or classes, and finer-grained program elements such as methods and fields, connected through relations that include folder containment, file-to-member composition, and static code-level dependencies such as calls, inheritance, type usage, and field access.

To preserve these distinctions, we represent each software system as a heterogeneous software graph $G = (V, E, \mathcal{A}, \mathcal{R})$. Here, V denotes the set of software artifact nodes and E denotes the set of relations between them. Each node $v \in V$ is assigned a node type through the mapping function $\tau(v) : V \to \mathcal{A}$, where $\mathcal{A}$ may include artifact types such as folders, classes, methods, and fields. Similarly, each edge $e \in E$ is assigned a relation type through $\phi(e) : E \to \mathcal{R}$, capturing folder hierarchy, the association between folders and classes, the association between classes and finer-grained program elements, and static code dependencies such as method calls, inheritance, and imports.

In this graph, the meaning of a relation is not characterized by the relation type alone. It also depends on the types of the source and target nodes that the edge connects. Therefore, for an edge $e_{s,t}$ from source node s to target node t, we describe the typed interaction as a meta-relation $\langle \tau(s), \phi(e_{s,t}), \tau(t) \rangle$. For example, a $\langle$folder, $contains$, file$\rangle$ relation captures project organization, whereas a $\langle$file, $Import$, file$\rangle$ relation captures code-level coupling. Thus, the same graph can preserve both structural organization and implementation dependencies without treating them as identical relations.

The Heterogeneous Graph Transformer (HGT) [14] is designed to exploit such meta-relations during message passing. It conditions attention and message transformation on the source node type, edge type, and target node type, making it suitable for code-to-architecture mapping, where different software relations provide different evidence for module assignment. Type-aware message passing determines how information flows across the graph, but the content that flows depends on how each node is represented before message passing begins, so the choice of node feature representation is not incidental to the graph design. Different feature choices encode different sources of evidence, and it is not obvious in advance which one a mapping model should rely on.

A lexical embedding trained on identifier tokens captures naming conventions but not a file's position within the dependency structure or the syntactic and behavioral content of its implementation. An embedding trained directly on the dependency graph, such as Node2Vec [53], instead encodes a file's structural neighborhood without reference to its content, and such embeddings have been shown to capture dependency-based cohesion effectively in architecture recovery [4, 11].

Pretrained code encoders such as UniXcoder [15] address the content gap left by lexical features by learning general-purpose representations of source code from large corpora, but their pretraining objective is not specific to architectural grouping: two files can be lexically and syntactically similar while belonging to different modules. Contrastive learning offers a way to adapt such an encoder to a task-specific notion of similarity by training the model to place designated positive pairs closer in the embedding space than unrelated in-batch examples, typically through an InfoNCE objective [54]. Positive pairs drawn from the static dependency graph rather than from lexical or structural similarity yield an embedding space that reflects dependency proximity.

IV. Methodology

This section presents the methodology for studying how architectural signals and graph design choices affect code-to-architecture mapping. We formulate the mapping task as semi-supervised node classification over a heterogeneous software graph $G = (V, E, \mathcal{A}, \mathcal{R})$, where $\mathcal{A}$ and $\mathcal{R}$ denote the sets of node and relation types. Let $V_f \subseteq V$ be the set of file nodes, which are the targets of the mapping task. A small subset $L \subset V_f$ is initially labeled with known architectural module assignments, and the remaining file nodes $U = V_f \setminus L$ are unlabeled. The objective is to learn an encoder:

$$Z = f_\theta(G), \qquad Z \in \mathbb{R}^{|V_f| \times d}, \tag{1}$$

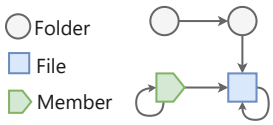
that maps each source file $i \in V_f$ to a d-dimensional embedding z_i, where θ denotes the learnable parameters of the representation-learning method. These file embeddings are used to predict architectural modules and to infer the labels of the nodes in U.

We first construct heterogeneous software graphs that vary folder organization, member-level information, and static dependencies. We next describe the structural, lexical, and contextual node feature initializations, the feature-only MLP, the HGT encoder, and the iterative learning procedure used to evaluate their contributions to mapping performance. The following configurations address the three research questions. For RQ1, we compare the HGT-based model on the complete folder–file–member graph against existing graph-based approaches. For RQ2, we compare structural, lexical, and contextual embeddings using the feature-only MLP, which classifies files without learned message passing. For RQ3, we use the same embeddings to initialize HGT across the graph variants in Section IV-A3 and compare each result with its feature-only counterpart.

A. Heterogeneous Graph Construction

We model the software system as a heterogeneous graph whose nodes represent software artifacts and edges represent organizational, containment, and dependency relations. The graph is built from three sources of information: the folder structure of the project, the containment of program elements within a file, and static dependencies extracted from source code. Folder structure provides organizational context, containment

connects folders to the files they contain and members to the files they belong to, and static dependencies describe interactions between software entities. We preserve these sources as distinct node and relation types rather than collapsing them into a single homogeneous graph. Figure 1 illustrates the general schema of the heterogeneous software graph and the meta-relations used in our model.



(a) Software graph schema

Folder	*parent_of*	Folder
Folder	*contains*	File
File	*depends_on*	File
Member	*belong_to*	File
Member	*depends_on*	Member

(b) The meta relations

Figure 1: Schema and meta-relations of the heterogeneous software graph. The figure shows the most expressive representation, with folder, file, and member nodes. Simpler variants remove selected node types or aggregate member-level dependencies at the file level.

1) Node Types: We use three node types: folder, file, and member. Folder nodes represent directory levels in the project structure. File nodes represent source files and are the target nodes for code-to-architecture mapping. Member nodes represent member-level program information associated with a file, including methods, fields, constructors, and variables.

We use one summarized *Member* node per file rather than representing each method, field, constructor, or variable as a separate node. The member node aggregates lexical and dependency information from the members of its associated file, retaining member-level information while avoiding the large and sparse graphs produced by fully fine-grained representations.

2) Relation Types: The graph contains structural relations, including organization, containment, and dependency. Logical components are often implemented through namespaces or directory structures that group source code providing a particular functionality, and such structures reflect how developers organize files [55, 6]. To preserve this information, each directory level is represented as a separate folder node. Folder nodes are connected through *parent_of* edges, forming hierarchical chains such as `app` → `app/ui` → `app/ui/views`. File nodes are linked to their folder through *contains* edges, and each member node is connected to its enclosing file through a *belongs_to* edge.

Static dependencies provide direct evidence of how entities interact and are a central source of information for architecture mapping and recovery [4, 56]. We extract them with the Depends tool [57], which analyzes the syntactic structure of the projects studied in this paper and identifies entities together with several categories of static relations. Different dependency types can carry distinct architectural meanings [5, 7], so we keep them as separate relational edge types rather than merging them into a single dependency relation. Table I lists the structural relations introduced during graph construction and the static dependency types observed in the analyzed Java and C/C++ systems.

Table I: Structural and static dependency relation types used in the software graph.

Relation type	Description
parent_of	Folder contains a subfolder in the directory.
contains	Folder contains a source file.
belongs_to	Member-level program element belongs to a file.
`Call`	Function or method invocation.
`Cast`	Conversion of an entity to another type.
`Contain`	Definition or containment of variables/fields.
`Create`	Creation of an instance or program entity.
`Extend`	Inheritance or parent-child type relation.
`Implement`	A class implements an interface or abstraction.
`Import`	A file imports other classes, methods, or variables.
`Parameter`	Use of a type as a method or function parameter.
`Return`	Use of a type as a return type.
`Use`	Use or assignment of variables.

`Annotation` and `Throw` are not observed in the analyzed C/C++ systems.

3) Graph Variants: Folder organization, member-level information, and static dependencies are available simultaneously in the complete graph, so their contributions cannot be determined from the full configuration alone. We therefore construct four graph variants that vary the presence of folder and member nodes while retaining dependency information at the available granularity, as summarized in Table II.

We distinguish dependencies by the granularity of their resolved endpoints: a dependency is file-level if at least one endpoint resolves to a file, which includes file–file, file–member, and member–file interactions, and member-level if both endpoints resolve to members. In FFM, dependency edges are modeled separately from the containment relations, so that file-level and member-level dependencies are preserved as distinct typed edges according to their resolved endpoints. FM removes folder nodes and folder relations but keeps file and member nodes, which isolates what remains recoverable from member-level information and dependencies when the project hierarchy is unavailable. FF and F remove member nodes; any member-level dependency is projected onto the file level by replacing each member endpoint with its containing file, so that all retained edges become file-level.

Table II: Graph variants used to study the effect of graph granularity and information sources.

Variant	Node types	Purpose
FFM	Folder, File, Member	Complete heterogeneous structure
FF	Folder, File	Folder and file-level structure
FM	File, Member	Member-level structure without folders
F	File	File-level dependency baseline

B. Node Feature Initialization

We initialize node features based on the information source used in each experimental setting. Folder nodes are always initialized from binary folder-path tokens, regardless of which strategy is used elsewhere in the graph. File and member nodes are initialized using one of three strategies: lexical embeddings from Word2Vec, structural embeddings from node2vec, or

contextual embeddings from a dependency-guided, fine-tuned UniXcoder, described below.

1) Folder Node Features: Folder nodes are encoded using a binary bag-of-words representation over folder tokens. The folder tokens are derived from directory segments after removing common project-root tokens. Each folder node is represented by the tokens along its path in the directory hierarchy. For example, a folder corresponding to `app/ui/views` carries the tokens `app`, `ui`, and `views`. This allows deeper folder nodes to retain information about their hierarchical context.

2) Word2Vec: Lexical Features : Source code identifiers often encode domain concepts and implementation roles, making them useful cues for mapping entities to architectural modules [7]. We use Word2Vec embeddings following the approach of Jabeen et al. [9] for lexical input. Tokens are extracted from file names and member-level program elements, including methods, fields, constructors, and variables. We normalize identifiers by splitting camelCase, PascalCase, and snake_case forms, converting tokens to lowercase, and removing stopwords and non-alphabetic tokens. The remaining tokens are represented using 128-dimensional skip-gram Word2Vec embeddings [58], trained for 20 epochs with a context window of five. Each node representation is computed by averaging the embeddings of its associated tokens.

Feature assignment depends on the graph variant. In FFM and FM, file nodes are initialized only from file-name tokens, while member nodes are initialized from tokens extracted from their respective program elements. In FF and File-only, member nodes are absent; therefore, member-level tokens are aggregated into the representation of the corresponding file node. This preserves the lexical information available across matched graph variants. Consequently, comparisons between FFM and FF and between FM and File-only isolate the effect of representing member-level information as separate nodes rather than incorporating the same information directly into file-node features.

3) Node2Vec: Structural Features : Chen et al. [4] observe that files in the same architectural module can occupy a dense local dependency region without being directly connected. Shallow message passing over raw dependency graphs can therefore provide too limited a context to expose such relationships, whereas random walks can capture longer-range dependency structure [11]. We use node2vec's [53] random-walk embeddings as a structural initialization and merge all static dependency types into a single edge type before generating random walks. Its biased random walks use parameters p and q to balance local and exploratory walks. The resulting embeddings capture structural similarity from dependency position without using identifier text or folder location.

In FM and FFM variants, member nodes carry the same node2vec embedding as their corresponding file node, rather than a separate embedding computed from a member-level dependency graph; a member-level graph would be far larger and sparser than the file-level graph, making random-walk-based embedding impractical at that granularity.

4) UniXcoder: Contextual Features : For contextual initialization, we use UniXcoder [15], a pretrained code transformer, in two variants: a zero-shot variant that encodes inputs directly with the pretrained weights, and a dependency-guided fine-tuned variant that adapts the encoder to each system's own dependency structure. Source files are parsed with tree-sitter [59] into separate text views for file and member nodes, aligning the inputs with the graph granularity. The file view contains the class header, description, and constructor signature, while the member view concatenates all method declarations of the class, excluding constructors. Building the views from parsed structure rather than raw text keeps the input compact and excludes comments and formatting from dominating the representation.

Jabeen et al. **(author?)** [11] show that shallow message passing over the raw dependency graph struggles to expose relationships between files that are connected only through shared neighbors rather than a direct edge. Therefore, positive pairs for contrastive fine-tuning are drawn primarily from the two-hop neighborhood: for each anchor file, two-hop candidates are ranked by the number of distinct one-hop neighbors shared with the anchor, and two pairs per file are sampled from the top eight candidates; files without any two-hop candidates fall back to a direct neighbor. Training uses an InfoNCE [54] objective with in-batch negatives; in-batch items connected by dependencies but not selected as a positive pair are masked so that they act as neither positive nor negative.

Only the upper four encoder layers are updated during fine-tuning, while the token embedding layer and the lower eight encoder layers stay frozen, since the number of dependency pairs available in a single system is small relative to the encoder's parameter count. Adaptation is performed independently for each subject system, as dependency structure and naming conventions are project-specific.

C. Mapping Models

1) HGT-Based Model: The HGT-based mapping model consists of a type-specific input projection, an HGT layer [14], and a file-level prediction head. Module assignments are predicted only for file nodes, while folder and member nodes provide context when present. Folder nodes supply organizational information, and member nodes supply member-level code information, and both reach file representations through message passing.

Node types have different feature spaces depending on the graph variant and the initialization strategy, so each type is first mapped into a shared hidden space with its own transformation as follows:

$$H_\tau^{(0)} = \mathrm{ReLU}(X_\tau W_\tau + b_\tau), \tag{2}$$

where X_τ is the input feature matrix of node type τ and W_τ and b_τ are type-specific learnable parameters, and ReLU is a non-linear activation function. This projection lets node types with different feature spaces be combined during message passing.

The HGT layer updates node representations through type- and relation-aware attention. For a target node t and a source node $s \in \mathcal{N}(t)$ connected by an edge $e = (s, t)$, the attention weight α and the transmitted message M are computed as:

$$\alpha(s,e,t) = \underset{s\in\mathcal{N}(t)}{\text{softmax}} \left(\frac{K_{\tau(s)}(h_s^{(0)}) W_{\phi(e)}^{ATT} \left(Q_{\tau(t)}(h_t^{(0)}) \right)^\top}{\sqrt{d}} \cdot \mu \right),$$
$$M(s,e,t) = M_{\tau(s)}(h_s^{(0)})\, W_{\phi(e)}^{MSG}, \tag{3}$$

where $Q_{\tau(t)}$ and $K_{\tau(s)}$ are type-specific query and key transformations, $W_{\phi(e)}^{ATT}$ and $W_{\phi(e)}^{MSG}$ are edge-type-specific matrices, μ is a learnable prior for the meta-relation $\langle \tau(s), \phi(e), \tau(t) \rangle$, and d is the dimensionality of one attention head. Multiple attention heads let the model attend differently across the graph's relation types within a single layer, which is what allows a *contains* edge from a folder and a `Call` edge from a dependent file to be weighted differently.

The attention-weighted messages are aggregated and combined with the projected input through a residual connection followed by layer normalization as follows:

$$h_t^{(1)} = \text{LayerNorm} \left(\sum_{s\in\mathcal{N}(t)} \alpha(s,e,t) \cdot M(s,e,t) + h_t^{(0)} \right). \tag{4}$$

The residual path preserves the initial representation of a node, which matters when a file's own feature-based evidence may be more reliable than a noisy neighborhood [60].

The file-level prediction head takes the final file-node representations $Z_f = H_f^{(1)}$ and produces module logits through a single linear layer, $\hat{Y}_f = Z_f W_c + b_c$, where W_c and b_c are learnable weight and bias parameters. The model is trained by minimizing categorical cross-entropy over the labeled file nodes.

2) Neural Baseline: The neural baseline is a multilayer perceptron (MLP) that predicts module logits directly from a file-level input vector, without the type-projection or message-passing steps used by the HGT-based model. Each hidden layer applies:

$$h_i^{(k+1)} = \text{ReLU}\big(W^{(k)} h_i^{(k)} + b^{(k)}\big), \tag{5}$$

with $h_i^{(0)} = x_i$ the file feature vector and $W^{(k)}, b^{(k)}$ the learnable parameters of layer k. After the final hidden layer, a linear layer maps the resulting representation to module logits. The same architecture is used throughout, with x_i varying by input configuration: a single feature type in isolation or a concatenation with folder-path features.

D. Iterative Learning Framework

The HGT-based model and the MLP baseline differ in architecture: HGT reaches a file's representation through type- and relation-aware message passing over the graph, whereas the MLP relies on the file's own feature vector alone. Regardless, both are trained with the same self-training procedure, applied identically across every graph variant and feature initialization in this paper.

Starting from the labeled set L, the model first trains in a supervised warm-up phase on L. It then scores every file node with a softmax over the predicted modules; the highest probability is the model's confidence in that prediction. In each self-training round, unlabeled file nodes whose confidence exceeds a threshold δ receive pseudo-labels and are added to L_{pseudo}, and the model is retrained on the union of the original and pseudo-labeled nodes. If a later round predicts a different label for a pseudo-labeled node with confidence above δ, the pseudo-label is updated. The procedure stops when no new confident nodes are found, when fewer than five nodes were added in the previous round, or when the maximum number of self-training rounds is reached. Algorithm 1 summarizes this loop.

Algorithm 1: Iterative Mapping Algorithm.

Input: Input representation $\mathcal{I}$, file nodes $\mathcal{F}$, labeled set L, confidence threshold δ, warm-up epochs E_w, self-training epochs E_s, max rounds R
Output: Trained classifier f, pseudo-labeled set L_{pseudo}

```
L_pseudo ← ∅; U ← F \ L;
Train f on (I, L) for E_w epochs;
added ← ∞;
for k ← 1 to R do
    if added < 5 then
        break;
    Score all files in F with f using I;
    Revise any pseudo-label in L_pseudo whose confidence ≥ δ
      disagrees with f;
    P ← {o ∈ U | confidence(o) ≥ δ};
    if P = ∅ then
        break;
    added ← |P|;
    L_pseudo ← L_pseudo ∪ P; U ← U \ P;
    Retrain f on (I, L ∪ L_pseudo) for E_s epochs;
return f, L_pseudo;
```

V. Experiments

This section describes the experimental design used to answer the three research questions: the benchmark systems, configuration, baselines, and evaluation metrics.

A. Benchmark Systems

We evaluate the HGT-based model on ten open-source systems with available ground-truth (GT) mappings. The systems have been used in prior work on software architecture recovery and C2A mapping [61, 9, 44]. They cover different programming languages, domains, sizes, and architectural granularities. Using published GT mappings is important because producing reliable C2A mappings requires substantial manual effort, architectural knowledge, and often expert validation [6]. These established mappings also support comparability with previous studies.

Table III summarizes the main system and graph properties used in our experiments. We also report file-level dependency homophily $\mathcal{H}$, computed as the proportion of static dependencies whose source and target files fall within the same GT module, ranging from 0, when no dependency stays within a module, to 1, when every dependency does.

Table III: Properties and versions of the subject systems. $\mathcal{C}$ denotes the number of modules in the GT architecture. $|E|$, $\mathcal{H}$, and $\langle k \rangle$ are computed on the file-level dependency graph, where $\langle k \rangle = |E|/|\text{Files}|$.

	System	Version	LoC	$\mathcal{C}$	Files	$\lvert E \rvert$	$\mathcal{H}$	$\langle k \rangle$
Java	A.UML	r13713	62k	14	766	9 491	0.54	12.4
	C.Img	v1.0a2	18k	21	329	4 561	0.61	13.9
	JabRef	v3.7	59k	6	1 180	17 557	0.58	14.9
	Lucene	r1075001	36k	7	1 054	18 595	0.68	17.6
	SH-3D	r002382	35k	9	191	3 912	0.34	20.5
	T.Mates	v5.11	55k	15	778	17 818	0.33	22.9
C	Bash	4.2	115k	14	292	1 222	0.60	4.2
	HDF	0e196f	153k	9	153	1 220	0.35	8.0
C++	Chrome	svn-171054	11M	69	18 343	284 435	0.58	15.5
	HDC	46ff87	26k	11	207	1 117	0.63	5.4

The Java systems are drawn from the benchmark used by Olsson et al. [44]. JabRef and TeamMates (T.Mates) were studied at the Software Architecture Erosion and Architectural Consistency Workshop (SAEroCon) in 2016 and 2017, respectively, where system experts provided the architectures and mappings[2]. ArgoUML (A.UML) and Lucene were studied by Brunet et al. [61] and Lenhard et al. [62], respectively. The Commons Imaging (C.Img) mapping was contributed as a test case for s4rdm3x [63]. The labels for all Java systems are available through the s4rdm3x repository[3].

For Bash, we use the C2A mapping curated by Garcia et al. [6]. Researchers recovered an initial decomposition from documentation and source code and refined it with system experts through a certification process. The same work provides the GT mapping for Chromium, developed through collaboration with developers over approximately two years [56].

The HDF and HDC mappings were developed with industry partners and are included in the SARIF benchmark studied by Zhang et al. [7]. They are available in the SARIF replication package[4].

B. Baseline Methods

For RQ1, we compare HGT with the graph-free MLP and relation-typed GAT baselines from the earlier C2A mapping study [**?**]. That study compared MLP, GCN, and GAT under the same features and iterative learning framework. GAT achieved the strongest overall performance, whereas GCN did not consistently outperform MLP; we therefore report MLP and GAT only.

GAT operates on a homogeneous file-only dependency graph with relation-specific message passing, whereas the MLP predicts module assignments without message passing. Both use concatenated ℓ_2-normalized Word2Vec identifier embeddings and binary bag-of-words folder-path features. This RQ1 MLP differs from the feature-only MLP in Section IV-C2, which receives only the single initialization embedding under evaluation and no folder-path features.

[2]https://github.com/sebastianherold/SAEroConRepo

[3]https://github.com/h0bb3/s4rdm3x

[4]https://github.com/anonymous2f4a9d/SARIF_FSE23

C. Experimental Configuration

We evaluate all neural methods in a low-label semi-supervised setting over 100 runs. For each run, the initial labeled set is constructed by randomly selecting 5% of file nodes, with a minimum of 30 and a maximum of 200 labeled files to avoid very small or disproportionately large training sets. If a GT module is absent from the sampled set, we add one file from that module so that all modules are initially represented. The same sampled initial set is used for all neural methods and configurations within each run. All neural models use the iterative self-training procedure described in Section IV-D.

Node2Vec embeddings are trained on the undirected file-level dependency graph using random walks of length 30, with 200 walks per node, $p = q = 1$, a context window of 15, and five training epochs with skip-gram and five negative samples, producing 128-dimensional embeddings. For Chromium, we use 100 walks per node and one negative sample. Embeddings are computed once per system and reused across runs.

UniXcoder fine-tuning uses an InfoNCE temperature of 0.05, AdamW with a learning rate of 2×10^{-5}, a batch size of eight, and two epochs with a fixed random seed. File-level inputs are encoded in a single pass with a 512-token limit. Member-level inputs are encoded in overlapping 256-token windows with a stride of 128 and mean-pooled before normalization, producing 768-dimensional node features.

For graph structures and node feature initializations, we follow the configurations described in Sections IV-A and IV-B. HGT uses one layer with 256 hidden dimensions and four attention heads. The feature-only MLP, the RQ1 MLP baseline, and GAT use two layers with 256 hidden dimensions. All neural models are trained with Adam at a learning rate of 0.001, using 100 supervised warm-up epochs and a confidence threshold of $\delta = 0.95$ for pseudo-label acceptance. HGT and the feature-only MLP use three and four self-training rounds, respectively; the RQ1 MLP and GAT use five rounds. Each self-training round consists of 30 epochs.

D. Performance Evaluation

The benchmark systems exhibit imbalanced module distributions, with some modules containing many files while others contain only a few. Accuracy can therefore overstate performance by giving disproportionate weight to the largest modules. Therefore, we evaluate mapping performance using the F1 score, which balances precision and recall. Precision is the proportion of predicted module assignments that are correct, and recall is the proportion of true assignments that are recovered, and F1 is their harmonic mean, appropriate when module sizes are imbalanced.

We report F1 using both macro- and micro-averaging. Macro-F1 is computed independently for each architectural module and then averaged, giving equal weight to every module regardless of size. It is therefore sensitive to performance on small or underrepresented modules. Micro-F1 aggregates predictions across all files before computing F1, providing an overall measure that gives greater weight to larger modules. Reporting

both metrics distinguishes class-balanced mapping performance from overall file-level performance.

VI. Results

This section reports the experimental results for semi-supervised code-to-architecture mapping across the subject systems, organized around three questions: whether representing software artifacts as a heterogeneous graph improves mapping performance over existing graph-based approaches (RQ1), which node feature type, structural, lexical, or contextual, is most informative for mapping (RQ2), and whether that relative effectiveness persists once the same features initialize a heterogeneous graph model, and how much graph structure adds beyond each feature type alone (RQ3). We report macro- and micro-averaged F1 scores computed over 100 runs with different initial labeled sets. All scores are computed over the complete test set, with no file excluded regardless of prediction confidence, so results reflect performance on every file a method is asked to map rather than only on those it maps with high confidence.

The initial labeled set is randomly sampled, and neural training introduces stochastic variation. Therefore, each experimental setting is repeated 100 times, and for each run, the same randomly sampled initial labeled set is used across all compared configurations. We report the median performance across runs because it is less sensitive to outliers than the mean, and we use the median F1 difference to quantify the practical performance gap between methods or configurations.

We assess the statistical significance of performance differences using the Wilcoxon signed-rank test [64] on macro-F1 and micro-F1 distributions across 100 runs, pairing runs by their shared random seed across compared configurations. To quantify these differences, we report Cliff's δ [65] as an effect size, interpreted using the standard convention: negligible (N, $\delta < 0.147$), small (S, $0.147 \leq \delta < 0.33$), medium (M, $0.33 \leq \delta < 0.474$), and large (L, $\delta \geq 0.474$). All reported p-values are Holm-corrected within each comparison family.

A. Effect of Heterogeneous Graph Representation (RQ1)

We compare HGT on the full folder-file-member graph (Section IV-A3) against the MLP, GCN, and GAT baselines (Section V-B). Unlike the baselines, whose single file-level node carries the full concatenated feature vector, HGT's file node does not carry code identifiers, which are instead represented on the member node; the file node is initialized with Word2Vec over the file name, concatenated with the same binary folder-path tokens used by the baselines. Table IV reports median macro-F1 and micro-F1 per system, and Table V reports the significance of the pairwise differences between HGT and each baseline.

HGT achieves the highest median macro-F1 on seven of the ten systems and the highest micro-F1 on eight. The gains over MLP are statistically significant with large effect sizes on most systems for both metrics. Against GAT, HGT is significantly better with large effects on seven systems, not significantly different on JabRef and Bash, and significantly worse on HDC.

Figure 2 shows the distribution of micro-F1 across the 100 runs for each system. On C.Img, T.Mates, and SH-3D, HGT's interquartile range is near zero, indicating that the heterogeneous representation produces stable results on these small, well-connected systems. Chrome's interquartile range is also narrow, though not as tight as these three, indicating comparable stability at a much larger scale. Bash, HDC, and HDF show the widest interquartile ranges in the benchmark for all methods. Together, these results show that the heterogeneous representation, combining structural, lexical, and organizational signals, achieves strong and largely consistent mapping performance across most systems. RQ2 isolates the contribution of the three node feature types, structural (dependency-graph-derived), lexical, and contextual, on their own, and RQ3 examines how organizational signal and heterogeneous message passing change that contribution when added on top of these features.

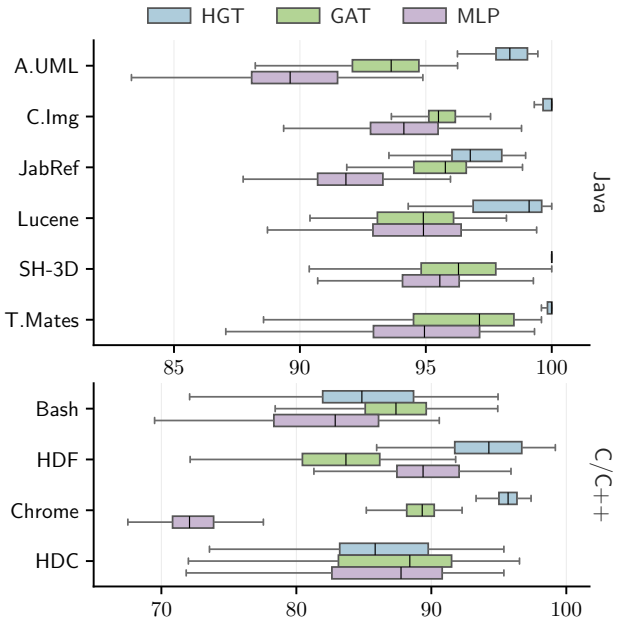


Figure 2: Micro-F1 distributions across 100 runs for each method on ten systems, grouped by language.

Answer to RQ1.

Representing software artifacts as a heterogeneous graph with typed nodes and relations improves mapping performance over relation-aware graph-based approaches (GAT) on most systems, with statistically significant gains and mostly large effect sizes; median macro-F1 gains reach 15 points over GAT and 26 points over MLP. The advantage is not universal: on small, less connected systems, HGT does not significantly separate from or is significantly outperformed by the baselines, indicating that the benefit of heterogeneous structure depends on the structural properties of the system.

Table IV: Median macro-F1 (MA) and micro-F1 (MI) scores (%) across 100 runs. MLP, GCN, and GAT use the concatenated feature vector described in Section V-B; HGT uses the FFM graph with Word2Vec-initialized file and folder nodes.

	A.UML		C.Img		JabRef		Lucene		SH-3D		T.Mates		Bash		HDF		Chrome		HDC	
Method	MA	MI	MA	MI	MA	MI	MA	MI	MA	MI	MA	MI	MA	MI	MA	MI	MA	MI	MA	MI
MLP	82	90	91	94	66	92	86	95	86	96	91	95	63	83	89	89	59	72	77	88
GAT	83	94	92	96	86	96	90	95	85	96	92	97	68	87	82	84	81	89	74	88
HGT	94	98	100	100	82	97	99	99	100	100	100	100	67	85	94	94	86	96	53	86

Table V: Median differences in F1 points for macro-F1 (MA) and micro-F1 (MI), with Cliff's δ magnitude in parentheses (N/S/M/L). † marks differences that are not statistically significant, and a positive value favors HGT. Cells where the baseline is significantly better than HGT are highlighted.

	HGT − GAT		HGT − MLP	
System	MA Δ	MI Δ	MA Δ	MI Δ
A.UML	+11 (L)	+5 (L)	+13 (L)	+9 (L)
C.Img	+8 (L)	+4 (L)	+9 (L)	+6 (L)
JabRef	−4 (N)†	+1 (M)	+16 (L)	+5 (L)
Lucene	+9 (L)	+4 (L)	+13 (L)	+4 (L)
SH-3D	+15 (L)	+4 (L)	+14 (L)	+4 (L)
T.Mates	+8 (L)	+3 (L)	+9 (L)	+5 (L)
Bash	−2 (N)†	−3 (S)	+4 (S)	+2 (S)
HDF	+12 (L)	+11 (L)	+5 (L)	+5 (L)
Chrome	+5 (L)	+6 (L)	+26 (L)	+24 (L)
HDC	−21 (L)	−3 (S)	−23 (L)	−2 (N)†

B. RQ2: Which file representation carries the most architectural signal?

RQ1 initialized every method with Word2Vec embeddings of identifiers or file names, concatenated with folder-path tokens, while also representing folder organization and member-level content explicitly in the graph structure. This leaves open which source of information carries the strongest architectural signal: a file's position in the dependency graph, its identifier vocabulary, or its implementation content. RQ2 addresses this question by comparing three representations of a file: a lexical representation based on Word2Vec (W2V) embeddings of identifiers; a structural representation based on node2vec (N2V) embeddings that encode a file's position in the dependency graph; and a contextual representation based on UniXcoder code embeddings in zero-shot (UXC_{ZS}) and dependency-guided fine-tuned (UXC_{FT}) variants. We train the MLP on each representation to classify modules. Table VI reports the median F1 scores for each system across 100 runs, and Table VII reports pairwise significance tests between representations.

Since the subject systems include small and imbalanced architectural modules, macro-F1 is generally lower and more variable than micro-F1. N2V achieves the highest median macro-F1 on seven systems, while UXC_{FT} leads on SH-3D, T.Mates, and HDC. For micro-F1, N2V leads on seven systems and UXC_{FT} on SH-3D and T.Mates; on JabRef, the two are tied and do not differ significantly. Where N2V significantly outperforms UXC_{FT}, the effect is large; UXC_{FT} significantly outperforms N2V with large effects on SH-3D and T.Mates. On HDC, UXC_{FT} is significantly better on macro-F1 with a large effect, whereas N2V is significantly better on micro-F1 with a small effect.

Fine-tuning significantly improves UniXcoder over UXC_{ZS} on macro-F1 for all ten systems and on micro-F1 for nine, mostly with large effects; the micro-F1 difference is not significant on Bash. Both N2V and UXC_{FT} significantly outperform W2V on both metrics for all ten systems, with large effects. The UXC_{ZS} representation also significantly outperforms W2V on both metrics for nine systems, except Chrome. These results confirm that W2V is the weakest representation among those compared.

Answer to RQ2.

Structural N2V and contextual UXC_{FT} provide the two strongest file representations. N2V achieves the highest median score on most systems, while UXC_{FT} is strongest on SH-3D and T.Mates. Fine-tuning significantly improves UniXcoder over its zero-shot counterpart, with large effects on nine of the ten systems. W2V is consistently weaker than N2V and UXC_{FT} on all systems, and weaker than UXC_{ZS} on nine systems, indicating that this lexical representation captures little architectural signal. Dependency structure underlies both leading representations: N2V encodes it directly, while UXC_{FT} is fine-tuned using dependency neighborhoods. The dependency graph is therefore the most consistently informative signal studied in RQ2.

C. RQ3: Node Feature Initialization and Its Dependence on Graph Structure

RQ2 established that N2V and UXC_{FT} are the two strongest representations without meta-relations and message passing, with W2V significantly weaker than both on nearly every system, and showed that this difference is attributable to dependency information. This leaves open whether that ranking persists once message passing over the dependency graph is introduced, and how much each representation gains from it. RQ3 addresses this by initializing HGT with W2V, N2V, and UXC_{FT} across five graph conditions: the graph-free MLP baseline; F and FM, which add dependency-based message passing without folder information; and FF and FFM, which add folder organization. UXC_{FT} is used exclusively in this section and is denoted UXC from here on. Unlike RQ1's HGT row, which appended folder-path tokens directly to file-node features, RQ3 initializes file nodes with each representation alone, so

Table VI: Median macro-F1 (MA) and micro-F1 (MI) scores (%) across 100 runs for the feature-only MLP baseline, evaluated with no graph structure and no message passing. Each row uses a different node feature type: structural (N2V), lexical (W2V), and contextual code embeddings from UniXcoder, both zero-shot (ZS) and dependency-guided fine-tuned (FT).

	A.UML		C.Img		JabRef		Lucene		SH-3D		T.Mates		Bash		HDF		Chrome		HDC	
Method	MA	MI	MA	MI	MA	MI	MA	MI	MA	MI	MA	MI	MA	MI	MA	MI	MA	MI	MA	MI
W2V	4	32	3	33	14	55	6	26	5	27	4	39	3	32	4	20	9	48	4	26
N2V	53	74	74	82	38	80	65	80	33	56	35	68	39	65	49	55	39	72	40	71
UXC_{ZS}	24	57	27	50	34	77	49	67	39	64	31	67	7	26	31	41	6	41	58	51
UXC_{FT}	35	64	40	63	37	80	59	76	42	67	49	79	4	28	40	49	10	52	69	66

Table VII: Median differences in F1 points for macro-F1 (MA) and micro-F1 (MI), with Cliff's δ magnitude in parentheses (N/S/M/L). † marks differences that are not statistically significant, and a positive favors the first-named method. Cells in which the second method is significantly better are highlighted.

	$UXC_{FT}-UXC_{ZS}$		$UXC_{FT}-W2V$		$UXC_{FT}-N2V$		N2V−W2V	
System	MA Δ	MI Δ	MA Δ	MI Δ	MA Δ	MI Δ	MA Δ	MI Δ
A.UML	+11.0(L)	+7.5(L)	+31.2(L)	+32.1(L)	−18.1(L)	−9.7(L)	+49.3(L)	+41.8(L)
C.Img	+13.4(L)	+13.8(L)	+36.9(L)	+30.6(L)	−33.9(L)	−19.0(L)	+70.8(L)	+49.6(L)
JabRef	+2.7(M)	+2.6(L)	+22.4(L)	+25.2(L)	−1.3(N)†	−0.3(N)†	+23.7(L)	+25.4(L)
Lucene	+10.4(L)	+8.5(L)	+53.5(L)	+49.9(L)	−5.3(M)	−4.4(L)	+58.9(L)	+54.3(L)
SH-3D	+2.4(S)	+2.7(M)	+36.9(L)	+39.4(L)	+9.0(L)	+10.3(L)	+27.9(L)	+29.1(L)
T.Mates	+17.4(L)	+11.8(L)	+44.7(L)	+40.6(L)	+13.8(L)	+11.0(L)	+30.9(L)	+29.6(L)
Bash	−2.5(L)	+1.8(N)†	+0.8(L)	−3.8(S)	−34.4(L)	−36.4(L)	+35.2(L)	+32.6(L)
HDF	+9.6(L)	+7.4(L)	+36.4(L)	+28.3(L)	−9.0(L)	−6.6(L)	+45.4(L)	+34.8(L)
Chrome	+3.9(L)	+10.7(L)	+1.3(L)	+4.1(L)	−28.5(L)	−19.9(L)	+29.9(L)	+24.0(L)
HDC	+10.8(L)	+15.4(L)	+65.3(L)	+39.9(L)	+29.2(L)	−5.0(S)	+36.1(L)	+44.9(L)

that FFM isolates the heterogeneous graph's contribution rather than combining it with a feature-level folder signal.

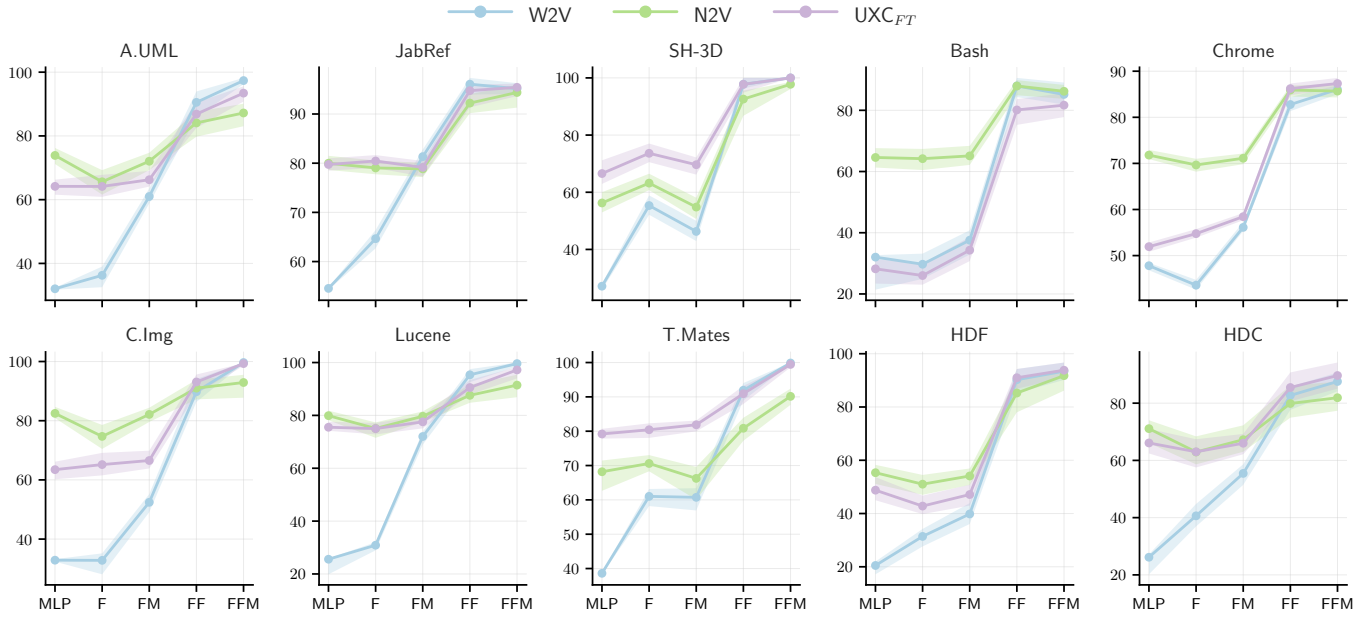
Figure 3 shows the full trajectory of each representation, per system, as graph structure is added incrementally from the graph-free MLP baseline through F, FM, FF, and FFM, for both micro- and macro-F1. Three patterns are visible across systems. W2V improves consistently as more structural information is added, increasing from the weakest initial performance to near-ceiling results under FFM. N2V is less consistent: it declines under F for several systems, partially recovers under FM, and remains close to its graph-free baseline elsewhere; substantial gains generally appear only after folder structure is introduced under FF and FFM. UXC_{FT} shows a similarly uneven pattern, with modest gains under F and FM and clearer improvements once folder structure is included. For HDF and HDC, however, performance declines under F and FM before recovering with folder-level information.

The shaded bands also indicate non-uniform variability across runs. Macro-F1 shows wider variation than micro-F1, particularly for JabRef and HDC, likely because a small number of sparsely labeled modules can substantially affect the macro average. Therefore, macro-F1 trends should be interpreted more cautiously than the accompanying micro-F1 results.

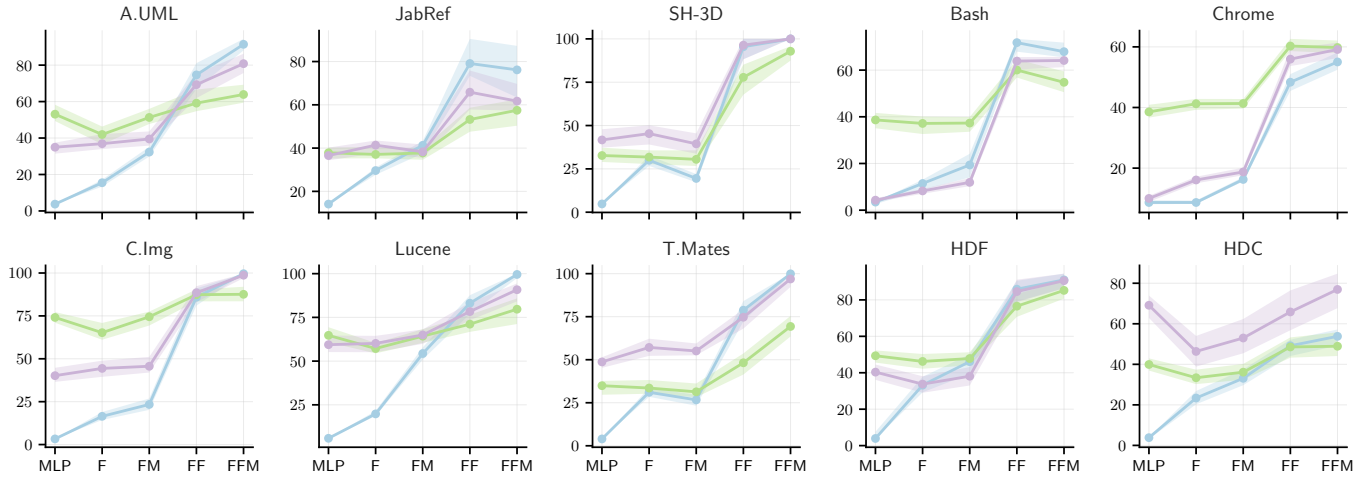
To assess whether the observed changes are statistically significant, Table VIII reports the effects of F and FM relative to each representation's graph-free MLP baseline, together with the direct FM − F comparison. The comparison focuses on F and FM because trajectories diverge most before folder structure is introduced and largely converge afterward. For W2V, dependency-based message passing yields large, significant gains on both metrics for most systems, reaching 48 macro-F1 points on Lucene under $W2V_{FM}$. Micro-F1 follows the same pattern, although the Bash gain is not significant, and F significantly underperforms the graph-free baseline on Chrome. This supports RQ2's finding that dependency structure is the most informative signal, and W2V does not encode it in its features, so it benefits most from message passing. For N2V, the macro-F1 pattern is largely reversed: message passing produces no significant gains and often small but significant decreases, consistent with N2V already encoding dependency structure. Micro-F1 is similar overall, although SH-3D and T.Mates gain significantly under F without a corresponding macro-F1 improvement. Chrome is the only system with a significant macro-F1 gain for N2V, despite significant micro-F1 decreases under both F and FM. UXC_{FT} improves on both metrics for most systems but declines substantially on HDF and HDC. Its largest decrease occurs on HDC under UXC_F, where macro-F1 falls by 23 points while micro-F1 declines by only 3 points. This difference is consistent with HDC's module-size skew, under which changes in a small number of sparsely labeled modules can disproportionately affect macro-F1.

The direct comparison of graph variants FM and F indicates that member-level structure is significantly beneficial for most systems and representations. SH-3D is the clearest recurring exception: FM significantly underperforms F for W2V, for N2V on micro-F1, and for UXC_{FT}. T.Mates shows the same reversal for W2V and N2V, while JabRef shows it for UXC_{FT} only.

Once the folder structure is added, the differences between representations largely disappear, as the convergence of the three lines under FF and FFM in Figure 3 shows. Under the



(a) Micro-F1



(b) Macro-F1

Figure 3: Median micro-F1 (top) and macro-F1 (bottom) for each representation, per system, across the graph-free MLP and HGT with increasing graph structure (F, FM, FF, FFM). Shaded areas denote interquartile ranges across the 100 runs.

full FFM graph, all three representations reach near-ceiling performance on most systems, and FFM is almost always the best or near-best condition regardless of initialization. The improvement from the graph-free baseline to FFM is large and consistent: W2V on Lucene rises from 6/26 to 100/100, N2V on SH-3D from 33/56 to 93/98, and UXC on T.Mates from 49/79 to 97/99. Because file nodes in FFM carry no folder-level feature, this improvement is attributable to the folder nodes and folder-file containment relations added to the graph, rather than to the initial representation. The systems where FFM does not reach near-ceiling performance, Bash and HDC, are also the systems where every condition struggles, indicating a limit in the available structural signal rather than in the choice of representation.

Folder location thus functions as a strong architectural signal in this benchmark, recoverable through message passing even when it is entirely absent from the file node's own features. RQ1 offers a second line of evidence for this: there, folder tokens are concatenated directly into the file node's own features, and HGT performs at or above its RQ3 FFM level on most systems, particularly on macro-F1, so folder information strengthens the representation whether it is supplied as a feature or recovered structurally through message passing. This strength should still be read with the caveat that folder hierarchies and the reference decompositions are shaped by related developer decisions, so folder structure may act in part as a proxy for the ground truth

Table VIII: Median differences in F1 points for macro-F1 (MA) and micro-F1 (MI), with Cliff's δ magnitude in parentheses (N/S/M/L). F − MLP and FM − MLP compare each graph condition with the graph-free MLP baseline for the same feature initialization; FM − F compares the two graph conditions directly. Positive values favor the first-named condition in each comparison. † denotes a non-significant difference. Highlighted cells indicate significant differences that favor the second-named condition.

		F − MLP		FM − MLP		FM − F	
	System	MA	MI	MA	MI	MA	MI
W2V	A.UML	$+12(L)$	$+4(L)$	$+29(L)$	$+29(L)$	$+17(L)$	$+25(L)$
	C.Img	$+13(L)$	$-0(N)^{\dagger}$	$+20(L)$	$+20(L)$	$+7(L)$	$+20(L)$
	JabRef	$+16(L)$	$+10(L)$	$+27(L)$	$+27(L)$	$+12(L)$	$+17(L)$
	Lucene	$+14(L)$	$+5(L)$	$+48(L)$	$+46(L)$	$+35(L)$	$+41(L)$
	SH-3D	$+25(L)$	$+28(L)$	$+15(L)$	$+19(L)$	$-10(L)$	$-9(L)$
	T.Mates	$+27(L)$	$+22(L)$	$+23(L)$	$+22(L)$	$-4(L)$	$-0(N)^{\dagger}$
	Bash	$+8(L)$	$-2(N)^{\dagger}$	$+16(L)$	$+6(L)$	$+8(L)$	$+8(L)$
	HDF	$+29(L)$	$+11(L)$	$+42(L)$	$+19(L)$	$+13(L)$	$+8(L)$
	Chrome	$+0(N)^{\dagger}$	$-4(L)$	$+8(L)$	$+8(L)$	$+8(L)$	$+13(L)$
	HDC	$+20(L)$	$+14(L)$	$+29(L)$	$+29(L)$	$+10(L)$	$+15(L)$
N2V	A.UML	$-11(L)$	$-8(L)$	$-2(S)$	$-2(S)$	$+9(L)$	$+6(L)$
	C.Img	$-9(L)$	$-8(L)$	$+0(N)^{\dagger}$	$-0(N)^{\dagger}$	$+9(L)$	$+7(L)$
	JabRef	$-1(N)^{\dagger}$	$-1(S)$	$-0(N)$	$-1(M)$	$+1(N)^{\dagger}$	$-0(N)^{\dagger}$
	Lucene	$-8(L)$	$-5(L)$	$-0(N)$	$-0(N)^{\dagger}$	$+7(L)$	$+5(L)$
	SH-3D	$-1(N)^{\dagger}$	$+7(L)$	$-2(S)$	$-1(S)^{\dagger}$	$-1(N)^{\dagger}$	$-8(L)$
	T.Mates	$-1(N)^{\dagger}$	$+3(M)$	$-4(S)$	$-2(S)$	$-2(S)^{\dagger}$	$-4(L)$
	Bash	$-2(S)$	$-0(N)^{\dagger}$	$-1(S)$	$+0(N)^{\dagger}$	$+0(N)^{\dagger}$	$+1(S)^{\dagger}$
	HDF	$-3(S)$	$-4(M)$	$-2(N)^{\dagger}$	$-1(N)^{\dagger}$	$+2(S)^{\dagger}$	$+3(S)$
	Chrome	$+3(L)$	$-2(L)$	$+3(L)$	$-1(S)$	$+0(N)^{\dagger}$	$+1(M)$
	HDC	$-6(L)$	$-8(L)$	$-4(M)$	$-4(S)$	$+3(S)$	$+4(M)$
UXC_{FT}	A.UML	$+2(S)$	$-0(N)^{\dagger}$	$+4(L)$	$+2(M)$	$+3(S)^{\dagger}$	$+2(S)^{\dagger}$
	C.Img	$+4(S)$	$+2(S)$	$+6(L)$	$+3(M)$	$+1(S)^{\dagger}$	$+1(S)^{\dagger}$
	JabRef	$+5(M)$	$+1(S)$	$+2(S)$	$-1(S)$	$-3(S)$	$-1(M)$
	Lucene	$+1(N)^{\dagger}$	$-0(S)^{\dagger}$	$+6(M)$	$+2(S)$	$+5(M)$	$+3(M)$
	SH-3D	$+4(S)^{\dagger}$	$+7(L)$	$-2(S)$	$+3(S)$	$-6(M)$	$-4(M)$
	T.Mates	$+8(L)$	$+1(S)$	$+6(L)$	$+3(L)$	$-2(N)^{\dagger}$	$+1(S)$
	Bash	$+4(L)$	$-2(N)^{\dagger}$	$+8(L)$	$+6(L)$	$+4(L)$	$+8(L)$
	HDF	$-7(L)$	$-6(L)$	$-2(S)^{\dagger}$	$-2(S)$	$+4(M)$	$+4(M)$
	Chrome	$+6(L)$	$+3(L)$	$+9(L)$	$+6(L)$	$+3(L)$	$+4(L)$
	HDC	$-23(L)$	$-3(S)$	$-16(L)$	$-0(N)^{\dagger}$	$+7(M)$	$+3(S)$

rather than as an entirely independent signal.

Answer to RQ3.

The relative effectiveness of the three representations does not persist once message passing is introduced. Message passing benefits each representation in proportion to how little dependency structure it already encodes: large gains for W2V, no gain or a small decrease for N2V in most cases, and mixed effects for UXC_{FT} that turn sharply negative on HDF and HDC. Once folder organization is added, this ranking no longer holds. All three representations reach near-ceiling performance on most systems under FFM despite carrying no folder-level feature, establishing folder location as the strongest architectural signal in this benchmark. Its reach is bounded: on Bash and HDC, no configuration approaches the ceiling, a limit set by the structural properties of these systems rather than by the choice of representation.

VII. Discussion

The three research questions examine where architectural signal resides in the software and how a mapping model should be built to use it. RQ1 establishes that a heterogeneous representation improves on the relation-aware GAT for most systems; RQ2 isolates which file representations carry the most architectural signal in the absence of message passing and organizational cues such as folder structure; and RQ3 traces how that signal changes as graph structure is progressively introduced, from dependency-based message passing alone to graphs that add member-level detail and folder organization. The discussion below considers the three sets of results together and identifies the conditions under which each observation holds.

Variability, system size and connectivity

The variability observed on Bash, HDF, and HDC in Figure 2 is best understood as a property of these systems rather than of any particular method. All approaches, including the graph-free MLP, show their widest interquartile ranges on these three systems, so the source of the spread is present before graph structure is introduced. These are among the smallest systems in the benchmark, which limits the size of the initial labeled set, and their comparatively granular architectures divide labeled files across more modules, leaving each module with fewer labeled files in a given run. When a module has few labeled files, its results depend strongly on which files are included in the initial sample, affecting both macro-F1 and micro-F1. Message passing does not reduce this variability on Bash, HDF, and HDC because these systems are also the least connected in the benchmark, with average degrees of 4.2, 8.0, and 5.4, respectively. With few dependency edges over which to propagate information, the graph has limited capacity to compensate for a small and unevenly distributed labeled set. SH-3D is a counterexample: it is also among the smallest systems in the benchmark, yet its average degree is 20.5, and its interquartile range is near zero for HGT and narrow for GAT. Thus, when connectivity is high, message passing and the structural signals it carries reduce variability even with a small initial training set, indicating that connectivity, rather than size alone, determines whether graph-based methods can compensate for label scarcity.

Homophily and the choice of file representation

RQ2 evaluates the structural, lexical, and contextual file representations without neural message passing, so the comparison reflects what the representation itself encodes. Given that some modules have few initially labeled files, we focus this comparison on micro-F1 because macro-F1 is more sensitive to performance variation in small modules. Node2Vec provides the strongest micro-F1 on most systems. Its performance is lower on the systems with the lowest dependency homophily (low cohesion and high coupling), T.Mates, SH-3D, and HDF, whose file-level homophily values are 0.33, 0.34, and 0.35, respectively. Bash is a partial exception: despite a homophily of 0.60, Node2Vec reaches a micro-F1 of only 65. Bash also has the lowest average degree in the benchmark, $\langle k \rangle = 4.2$, and all representations perform comparatively poorly on this system. Nevertheless, Node2Vec remains the strongest representation on Bash and performs significantly

better than lexical and contextual embeddings. Chrome shows that Node2Vec can remain effective under limited supervision and high architectural granularity. With just over one percent of files initially labeled and 69 ground-truth modules, Node2Vec reaches a micro-F1 of 72, significantly exceeding lexical and contextual representations.

Contextual embeddings, when fine-tuned on the dependency graph, offer a comparable structural signal through a different path. This representation outperforms node2vec on SH-3D and T.Mates, and also on HDC on macro-F1, indicating that dependency structure overall provides a strong architectural signal, but remains bounded by architectural cohesion when no other signals are present.

Impact of organizational signal

When folder-level information is available to HGT, either as folder tokens concatenated with the initial file features (RQ1) or as folder-organization edges connecting file nodes to folder nodes (RQ3), the model reaches near-ceiling performance on most systems. In RQ3, all three feature initializations converge on comparable results despite encoding different information at the file level. However, in the absence of folder information, this ranking instead varies with graph homophily and connectivity, as well as with the extent to which the code representations distinguish the modules of each system.

This pattern is particularly clear for Word2Vec. Although it provides the weakest architectural signal in the feature-only RQ2 comparison, it matches or exceeds Node2Vec and UXC_{FT} on most systems under the full FFM graph, with its advantage particularly pronounced on JabRef, T.Mates, and Bash. In FFM, a Word2Vec-initialized file node contains only an embedding of its file name; code identifiers are represented at the member node, while dependency information is represented by graph relations. Thus, its strong performance under FFM should not be interpreted as evidence that Word2Vec itself captures more architectural information than the other initializations. Rather, it indicates that the organizational information available through folder-file containment can substantially compensate for a comparatively weak file-level initialization.

Chrome and HDC are exceptions to this pattern, and both are consistent with the feature-only results in RQ2. On Chrome, Word2Vec trails Node2Vec by five macro-F1 points under FFM. Node2Vec already provides strong architectural information in the feature-only comparison and remains the stronger initialization in both the folder–file and FFM variants. On HDC, Word2Vec trails UXC_{FT} by 23 macro-F1 points. UXC_{FT} is also the strongest feature-only representation on HDC and improves further when folder structure is introduced, increasing from 69/66 macro-/micro-F1 without folder information to 77/90 under FFM. Folder organization, therefore, improves all three initializations, but it does not always remove performance differences when a representation already provides strong system-specific architectural information.

What does member-level structure add?

The RQ3 results show that the benefit of dependency-based message passing depends on how much architectural information the initial representation already carries. Word2Vec benefits most: RQ2 showed that its embeddings of code identifiers provide limited architectural signal, and this signal is available differently across graph conditions. In F and FF, the initial file representation includes both its name and code identifiers. In FM and FFM, code identifiers belong to member nodes instead, so the file node carries only its name embedding, which acts like an identity vector. HGT must therefore use dependency and containment meta-relations to transform this weak initial representation into a more structurally informed one, with member-to-file relations providing a finer-grained path for this information to reach the file. Node2vec often changes little or declines because its random-walk embeddings already encode dependency neighborhoods, making shallow message passing over the same edges largely redundant. Its exceptions occur on the low-homophily systems, SH-3D and T.Mates, where random-walk neighborhoods appear not to capture the relevant modular structure sufficiently, and message passing provides complementary local information. UXC_{FT} shows a pattern similar to Word2Vec: its contrastive fine-tuning provides dependency guidance but does not replace the one-hop information available through message passing. Accordingly, it improves under F and FM on most systems, with the two smallest and least connected systems, HDF and HDC, as exceptions.

Member-level structure helps only when the graph provides a sufficiently homophilic structure on which to build. When moving from F to FM, Word2Vec and UXC_{FT} improve on every system except SH-3D and T.Mates, the two systems with the lowest homophily and highest node degree, where the added member-level graph reduces performance for both. HDF also has low homophily but contains only two member-to-member edges under the FM and FFM modeling convention, too few to meaningfully affect message passing. JabRef is a further exception, and its coarse-grained architecture may explain why. Its module distribution is highly skewed: one module is much larger than the others, while three of its six modules are very sparsely represented. With most files concentrated in a single large module, files already interact heavily at the file level, leaving little extra information for member nodes to add and potentially diluting the direct file-level signal. Node2vec follows the same pattern, improving little or declining on SH-3D and T.Mates, which suggests that member-level structure is least reliable in heterophilic systems.

Word2Vec also achieves the highest FFM scores on most systems because its weak initial file representation leaves greater scope for structural information to shape the learned embedding. Once the folder-file containment meta-relation is available, HGT combines this strong organizational signal with member-level dependencies. Node2vec and UXC_{FT} also improve under FFM, but their initial file features already contain more architectural information. For UXC_{FT}, member-

level structure therefore serves a different purpose. Its file-level initialization combines the file name, description, and constructor, which carry some architectural signal, as zero-shot UXC_{FT} outperforms Word2Vec in RQ2. This initialization never includes member-level code structure under any graph condition, which becomes accessible only when member nodes are present. Member-level structure, therefore, allows HGT to construct additional architectural detail from these relations rather than compensating for an entirely weak file representation. Earlier experiments that encoded the full file AST directly into a single file-level embedding performed poorly, whereas keeping the file feature compact and letting HGT learn which member-level signals to propagate through typed message passing proved more effective.

Member-level modeling is therefore a conditional advantage rather than a universally beneficial addition. It is most useful when the initial representation lacks architectural information, the file-level graph is sufficiently homophilic, and member-level edges provide additional evidence beyond file and folder structure.

Where the pattern breaks down?

Bash and HDC are the systems in which the heterogeneous representation does not clearly outperform simpler baselines in RQ1 and where the full FFM graph does not reach near-ceiling performance in RQ3. As the least connected systems in the benchmark, their behavior is better explained by limited connectivity than by homophily. Moving from FF to FFM adds little: for Node2vec, performance is unchanged on HDC and declines on Bash, while for UXC_{FT} it is unchanged on Bash. With few dependency edges distributed across member nodes, the additional member-level structure provides little information to propagate. Thus, the more detailed graph does not improve on the coarser representation, suggesting that the limitation lies in the structural evidence exposed by the system rather than in feature initialization or model choice.

Synthesis: Folder organization is the strongest and most transferable signal in this benchmark: provided as either a lexical feature or a graph relation, it accounts for most of the improvement in mapping performance and largely reduces differences between node features. However, this strength requires caution, because folder placement and the reference decomposition may reflect related developer decisions; its advantage may therefore partly arise from proximity to the target labels rather than from an independent architectural signal. Dependency structure is the next most informative source of evidence, but its value is shaped by both connectivity and homophily. Sparse connectivity limits how much dependency context any representation can rely on, so the least connected systems gain the least from graph structure regardless of the initial feature. Homophily is the larger constraint specifically for Node2Vec: because it has no independent node feature and encodes the graph directly, its embedding degrades on the least homophilic systems even where connectivity is not the bottleneck, while attention-based message passing in HGT can partly compensate for low homophily elsewhere. This is also why message passing mainly benefits weak lexical initializations and adds less where a feature already encodes dependency neighborhoods, as Node2Vec does. Contextual code embeddings are most useful when the dependency structure is weakly aligned with reference modules. Overall, mapping performance depends less on the initial representation than on which structural signals are available and how well they align with the reference module decomposition.

VIII. Threats to Validity

External Validity: The evaluation covers ten open-source systems in Java, C, and C++ with available ground-truth mappings, which sample only part of the variation in size, domain, and architectural style found in practice, so the findings may not generalize beyond this benchmark. None of the systems consist largely of AI-generated code, and whether such code yields dependency structures with comparable module cohesion is untested. The relationships between performance and structural properties, notably homophily and average node degree, are observed across ten systems and are reported as associations rather than causal claims; the benchmark is not large enough to separate these properties from one another or from system size, which co-varies with connectivity.

Internal Validity: Dependencies are extracted with a static analysis tool, which may miss interactions arising from dynamic features, reflection, or incomplete parsing, and may introduce edges that do not reflect an actual dependency; the resulting graph can therefore under- or over-represent the true coupling between files, though this affects every representation and model equally. Because the initial labeled set is drawn from the same mapping used for evaluation, an annotation error can both mislead training and penalize a correct prediction. With 100 runs per configuration, statistical tests can detect small systematic differences that are not necessarily practically meaningful; we report effect sizes alongside significance to guard against this.

Construct Validity: The ground-truth module assignments used here are drawn from a single authoritative decomposition per system, so the reported scores measure agreement with one reference architecture rather than with an unarguable ground truth. For systems where developers themselves defined the architecture, folder hierarchies and reference architectures are shaped by the same organizational decisions; for systems where researchers reconstructed the architecture from documentation and implementation, folder structure is itself one of the cues such reconstructions typically rely on. In both cases, folder location is likely to align with the reference decomposition for reasons unrelated to its standing as an independent architectural signal, so its apparent predictive strength may partly reflect this alignment rather than a genuinely independent source of architectural evidence.

All reported scores are computed over every file in the test set, with no file excluded for low prediction confidence, so performance differences reflect mapping accuracy on the full evaluation set rather than a model's ability to abstain on uncertain cases. Macro-F1 is sensitive to systems with skewed

module sizes, as the HDC results show, so macro-level differences on such systems should be read alongside micro-F1. The study also scopes architectural signal to structural, lexical, and contextual file representations together with folder organization; other signals used in prior recovery and mapping work, such as commit co-change history and dynamic dependencies observed at runtime, are not included, and their contribution alongside the signals studied here remains open.

Finally, the confidence threshold for pseudo-labeling reflects how sharply a model's softmax output concentrates, not only how well it maps files, and module assignment is evaluated at file-level granularity, so a file spanning more than one architectural concern cannot be labeled accurately by any of the compared models.

IX. Conclusion

This paper examined where architectural signal resides in software systems and how a mapping model should represent it. We used a feature-only neural baseline and HGT to vary node features and graph structure independently. Across ten systems, node2vec provided the strongest mapping signal without message passing on most systems, followed closely by dependency-guided fine-tuned UniXcoder. Both significantly outperformed lexical Word2Vec embeddings. Fine-tuned UniXcoder outperformed its zero-shot counterpart, showing that the advantage of contextual embeddings arose from dependency-guided fine-tuning rather than from the zero-shot representation of code content. Thus, dependency-derived information was the most informative signal before message passing, whether encoded directly by node2vec or introduced through fine-tuning. This ranking changed when message passing was introduced. Dependency-based propagation benefited each representation in proportion to how little dependency structure it already encoded. Word2Vec, the weakest representation alone, gained the most because HGT could use dependency and containment relations to compensate for the limited architectural information in identifier-based embeddings. Once folder organization was added, the three representations converged to near-ceiling performance, despite carrying no folder feature at the file level.

Folder location was thus the strongest and most transferable signal in this benchmark, recoverable either as a node feature or through message passing over containment relations. Static dependencies remained useful, but their contribution was bounded by system connectivity and architectural cohesion. Member-level structure provided a more conditional source of evidence. It helped most when file-level representations were weak and the graph exhibited sufficient architectural cohesion, but contributed little when dependencies were sparse or already captured at the file level. These findings show that, once sufficient structural signal is available, mapping performance depends less on the initial file representation than on which structural signals a system exposes and how well they align with its reference decomposition.

Future work should extend this comparison to signals not studied here, such as commit co-change history and dynamic dependencies observed at runtime, and examine whether their contribution follows the same pattern found for the static signals in this paper. Disentangling folder organization's predictive strength from its overlap with how reference architectures are constructed, discussed in Section VIII, is a further direction, along with testing whether the ordering of signals established here holds on systems and reference architectures beyond those studied.

References

[1] P. Clements, D. Garlan, R. Little, R. Nord, and J. Stafford, "Documenting software architectures: views and beyond," in *25th International Conference on Software Engineering*. IEEE, 2003, pp. 740–741.

[2] D. L. Parnas, "On the criteria to be used in decomposing systems into modules," *Communications of the ACM*, vol. 15, no. 12, pp. 1053–1058, 1972.

[3] L. De Silva and D. Balasubramaniam, "Controlling software architecture erosion: A survey," *Journal of Systems and Software*, vol. 85, no. 1, pp. 132–151, 2012.

[4] J. Chen, Z. Wang, Y. Jiang, J. Pang, T. Zhang, M. Pan, and J. Sun, "NEGAR: network embedding guided architecture recovery for software systems," in *2022 29th Asia-Pacific Software Engineering Conference (APSEC)*. IEEE, 2022, pp. 367–376.

[5] I. Stavropoulou, M. Grigoriou, and K. Kontogiannis, "Case study on which relations to use for clustering-based software architecture recovery," *Empirical Software Engineering*, vol. 22, no. 4, pp. 1717–1762, 2017.

[6] J. Garcia, I. Krka, C. Mattmann, and N. Medvidovic, "Obtaining ground-truth software architectures," in *35th International Conference on Software Engineering (ICSE)*. IEEE, 2013, pp. 901–910.

[7] Y. Zhang, Z. Xu, C. Liu, H. Chen, J. Sun, D. Qiu, and Y. Liu, "Software architecture recovery with information fusion," in *Proceedings of the 31st ACM Joint ESEC/FSE*, 2023, pp. 1535–1547.

[8] A. Rathee and J. K. Chhabra, "Software remodularization by estimating structural and conceptual relations among classes and using hierarchical clustering," in *Advanced Informatics for Computing Research: ICAICR 2017*. Springer, 2017, pp. 94–106.

[9] R. Jabeen, M. Ericsson, J. Nordqvist, and A. Wingkvist, "GAER: Graph auto-encoders for unsupervised software architecture recovery," *Proceedings of the ACM on Software Engineering*, vol. 3, no. FSE, Jul. 2026.

[10] ——, "Graph convolution networks for mapping source code entities to architectural modules," in *2025 IEEE 22nd International Conference on Software Architecture Companion (ICSA-C)*. IEEE, 2025, pp. 472–481.

[11] ——, "Graph embeddings for software architecture recovery," *Nordic Machine Intelligence*, 2026. [Online]. Available: https://journals.uio.no/NMI/article/view/13153

[12] G. C. Murphy, D. Notkin, and K. Sullivan, "Software reflexion models: Bridging the gap between source and high-level models," in *3rd ACM SIGSOFT Symposium on Foundations of Software Engineering*, 1995, pp. 18–28.

[13] N. Ali, S. Baker, R. O'Crowley, S. Herold, and J. Buckley, "Architecture consistency: State of the practice, challenges and requirements," *Empirical Software Engineering*, vol. 23, pp. 224–258, 2018.

[14] Z. Hu, Y. Dong, K. Wang, and Y. Sun, "Heterogeneous graph transformer," in *Proceedings of the Web Conference 2020*, 2020, pp. 2704–2710.

[15] D. Guo, S. Lu, N. Duan, Y. Wang, M. Zhou, and J. Yin, "Unixcoder: Unified cross-modal pre-training for code representation," in *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2022, pp. 7212–7225.

[16] S. Mancoridis, B. S. Mitchell, Y. Chen, and E. R. Gansner, "Bunch: A clustering tool for the recovery and maintenance of software system structures," in *Proceedings IEEE International Conference on Software Maintenance (ICSM'99)*. IEEE, 1999, pp. 50–59.

[17] B. S. Mitchell and S. Mancoridis, "On the automatic modularization of software systems using the bunch tool," *IEEE Transactions on Software Engineering*, vol. 32, no. 3, pp. 193–208, 2006.

[18] S. Parsa and O. Bushehrian, "A new encoding scheme and a framework to investigate genetic clustering algorithms," *Journal of Research and Practice in Information Technology*, vol. 37, no. 1, pp. 127–143, 2005.

[19] O. Maqbool and H. A. Babri, "The weighted combined algorithm: A linkage algorithm for software clustering," in *Eighth European Conference*

on Software Maintenance and Reengineering (CSMR 2004). IEEE, 2004, pp. 15–24.

[20] M. Harman and X. Yao, "Software module clustering as a multi-objective search problem," *IEEE Transactions on Software Engineering*, vol. 37, no. 2, pp. 264–282, 2010.

[21] R. Naseem, O. Maqbool, and S. Muhammad, "Cooperative clustering for software modularization," *Journal of Systems and Software*, vol. 86, no. 8, pp. 2045–2062, 2013.

[22] R. Naseem, M. M. Deris, O. Maqbool, and S. Shahzad, "Euclidean space based hierarchical clusterers combinations: an application to software clustering," *Cluster Computing*, vol. 22, no. Suppl 3, pp. 7287–7311, 2019.

[23] P. Andritsos, P. Tsaparas, R. J. Miller, and K. C. Sevcik, "Limbo: Scalable clustering of categorical data," in *International Conference on Extending Database Technology*. Springer, 2004, pp. 123–146.

[24] N. Teymourian, H. Izadkhah, and A. Isazadeh, "A fast clustering algorithm for modularization of large-scale software systems," *IEEE Transactions on Software Engineering*, vol. 48, no. 4, pp. 1451–1462, 2020.

[25] S. Mohammadi and H. Izadkhah, "A new algorithm for software clustering considering the knowledge of dependency between artifacts in the source code," *Information and Software Technology*, vol. 105, pp. 252–256, 2019.

[26] B. Pourasghar, H. Izadkhah, A. Isazadeh, and S. Lotfi, "A graph-based clustering algorithm for software systems modularization," *Information and Software Technology*, vol. 133, p. 106469, 2021.

[27] J. Garcia, D. Popescu, C. Mattmann, N. Medvidovic, and Y. Cai, "Enhancing architectural recovery using concerns," in *2011 26th IEEE/ACM International Conference on Automated Software Engineering (ASE 2011)*. IEEE, 2011, pp. 552–555.

[28] A. Corazza, S. Di Martino, V. Maggio, and G. Scanniello, "Investigating the use of lexical information for software system clustering," in *2011 15th European Conference on Software Maintenance and Reengineering*. IEEE, 2011, pp. 35–44.

[29] M. Risi, G. Scanniello, and G. Tortora, "Using fold-in and fold-out in the architecture recovery of software systems," *Formal Aspects of Computing*, vol. 24, no. 3, pp. 307–330, 2012.

[30] M. Kargar, A. Isazadeh, and H. Izadkhah, "Semantic-based software clustering using hill climbing," in *2017 International Symposium on Computer Science and Software Engineering Conference (CSSE)*. IEEE, 2017, pp. 55–60.

[31] N. Anquetil and T. C. Lethbridge, "Recovering software architecture from the names of source files," *Journal of Software Maintenance: Research and Practice*, vol. 11, no. 3, pp. 201–221, 1999.

[32] J. Misra, K. Annervaz, V. Kaulgud, S. Sengupta, and G. Titus, "Software clustering: Unifying syntactic and semantic features," in *2012 19th Working Conference on Reverse Engineering*. IEEE, 2012, pp. 113–122.

[33] W. Mkaouer, M. Kessentini, A. Shaout, P. Koligheu, S. Bechikh, K. Deb, and A. Ouni, "Many-objective software remodularization using nsga-iii," *ACM Transactions on Software Engineering and Methodology*, vol. 24, no. 3, pp. 1–45, 2015.

[34] N. Sadat Jalali, H. Izadkhah, and S. Lotfi, "Multi-objective search-based software modularization: Structural and non-structural features," *Soft Computing*, vol. 23, no. 21, 2019.

[35] S. P. R. Puchala, J. K. Chhabra, and A. Rathee, "Ensemble clustering based approach for software architecture recovery," *International Journal of Information Technology*, vol. 14, no. 4, pp. 2013–2019, 2022.

[36] K. Yang, J. Wang, Z. Fang, P. Wu, and Z. Song, "Enhancing software modularization via semantic outliers filtration and label propagation," *Information and Software Technology*, vol. 145, p. 106818, 2022.

[37] M. Papachristou, "Software clusterings with vector semantics and the call graph," in *Proceedings of the 27th ACM Joint ESEC/FSE*, 2019, pp. 1184–1186.

[38] V. Tzerpos and R. C. Holt, "The orphan adoption problem in architecture maintenance," in *Working Conference on Reverse Engineering*. IEEE, 1997, pp. 76–82.

[39] A. Christl, R. Koschke, and M.-A. Storey, "Equipping the reflexion method with automated clustering," in *12th Working Conference on Reverse Engineering (WCRE'05)*. IEEE, 2005, pp. 10–pp.

[40] ——, "Automated clustering to support the reflexion method," *Information and Software Technology*, vol. 49, no. 3, pp. 255–274, 2007.

[41] R. A. Bittencourt, G. J. d. S. Santos, D. D. S. Guerrero, and G. C. Murphy, "Improving automated mapping in reflexion models using information retrieval techniques," in *17th Working Conference on Reverse Engineering*, 2010, pp. 163–172.

[42] M. Bibi, O. Maqbool, and J. Kanwal, "Supervised learning for orphan adoption problem in software architecture recovery," *Malaysian Journal of Computer Science*, vol. 29, no. 4, pp. 287–313, 2016.

[43] T. Olsson, M. Ericsson, and A. Wingkvist, "Semi-automatic mapping of source code using naive bayes," in *Proceedings of the 13th European Conference on Software Architecture*, 2019, pp. 209–216.

[44] ——, "To automatically map source code entities to architectural modules with naive bayes," *Journal of Systems and Software*, vol. 183, p. 111095, 2022.

[45] A. Florean, L. Jalal, Z. T. Sinkala, and S. Herold, "A comparison of machine learning-based text classifiers for mapping source code to architectural modules," in *15th European Conference on Software Architecture*, 2021.

[46] Z. T. Sinkala and S. Herold, "Inmap: Automated interactive code-to-architecture mapping recommendations," in *18th International Conference on Software Architecture*, 2021, pp. 173–183.

[47] ——, "An integrated approach to package and class code-to-architecture mapping using inmap," in *2023 IEEE 20th International Conference on Software Architecture (ICSA)*. IEEE, 2023, pp. 164–174.

[48] N. Johansson, M. Caporuscio, and T. Olsson, "Mapping source code to software architecture by leveraging large language models," in *European Conference on Software Architecture*. Springer, 2024, pp. 133–149.

[49] N. Johansson and M. Caporuscio, "Mapping source code to software architecture using embedding models: Insights from an analysis of data properties," *SSRN Electronic Journal*, 2025, preprint, SSRN 5798927. [Online]. Available: https://ssrn.com/abstract=5798927

[50] D. Fuchß, H. Liu, T. Hey, J. Keim, and A. Koziolek, "Enabling architecture traceability by llm-based architecture component name extraction," in *2025 IEEE 22nd International Conference on Software Architecture (ICSA)*. IEEE, 2025, pp. 1–12.

[51] S. A. Rukmono, L. Ochoa, and M. Chaudron, "Deductive software architecture recovery via chain-of-thought prompting," in *44th International Conference on Software Engineering: New Ideas and Emerging Results*, 2024, pp. 92–96.

[52] R. Jabeen, M. Ericsson, J. Nordqvist, and A. Wingkvist, "Graph representation learning for software architecture recovery," in *ESANN*, 2026.

[53] A. Grover and J. Leskovec, "node2vec: Scalable feature learning for networks," in *Proceedings of the 22nd ACM SIGKDD international conference on Knowledge discovery and data mining*, 2016, pp. 855–864.

[54] A. van den Oord, Y. Li, and O. Vinyals, "Representation learning with contrastive predictive coding," *arXiv preprint arXiv:1807.03748*, 2018.

[55] M. Richards and N. Ford, *Fundamentals of Software Architecture: A Modern Engineering Approach*. O'Reilly Media, 2025.

[56] T. Lutellier, D. Chollak, J. Garcia, L. Tan, D. Rayside, N. Medvidović, and R. Kroeger, "Measuring the impact of code dependencies on software architecture recovery techniques," *IEEE Transactions on Software Engineering*, vol. 44, no. 2, pp. 159–181, 2017.

[57] H. V. Nguyen, "Depends: Multilingual dependency analysis tool," https://github.com/multilang-depends/depends, 2023, accessed: 2025-08-17.

[58] T. Mikolov, I. Sutskever, K. Chen, G. S. Corrado, and J. Dean, "Distributed representations of words and phrases and their compositionality," *Advances in Neural Information Processing Systems*, vol. 26, 2013.

[59] M. Brunsfeld and contributors, "Tree-sitter: An incremental parsing system for programming tools," https://tree-sitter.github.io/tree-sitter/, 2018, accessed: 2026-04-22.

[60] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 770–778.

[61] J. Brunet, R. A. Bittencourt, D. Serey, and J. Figueiredo, "On the evolutionary nature of architectural violations," in *2012 19th Working Conference on Reverse Engineering*. IEEE, 2012, pp. 257–266.

[62] J. Lenhard, M. Blom, and S. Herold, "Exploring the suitability of source code metrics for indicating architectural inconsistencies," *Software Quality Journal*, vol. 27, no. 1, pp. 241–274, 2019.

[63] T. Olsson, M. Ericsson, and A. Wingkvist, "s4rdm3x: A tool suite to explore code to architecture mapping techniques," *Journal of Open Source Software*, vol. 6, no. 58, 2021.

[64] F. Wilcoxon, "Individual comparisons by ranking methods," *Biometrics bulletin*, vol. 1, no. 6, pp. 80–83, 1945.

[65] N. Cliff, "Dominance statistics: Ordinal analyses to answer ordinal questions," *Psychological bulletin*, vol. 114, no. 3, p. 494, 1993.